

처음부터 재현하는 방법

아무것도 없는 계정에서 이 평가를 그대로 돌리는 절차입니다.
데이터베이스도, 실행 중인 API 서버도 필요 없습니다. LLM 호출만 가능하면 됩니다.

1. 준비물

	내용
Python	3.13 이상
소스	SalesLuv 저장소 (backend/ 폴더가 필요합니다)
LLM	OpenAI 호출 엔드포인트와 키

2. 가상환경과 의존성

```
`bash
cd backend
python3 -m venv .venv
.venv/bin/pip install -e .
`

pyproject.toml 의 의존성을 그대로 씁니다. 별도 requirements 파일은 없습니다.
```

3. 환경변수

backend/.env 를 만들고 아래 값을 채웁니다. backend/.env.example 에 전체 목록이 있습니다.

```
APP_ENV=local
LLM_API_URL=
LLM_API_KEY=
LLM_MODEL=
LLM_TIMEOUT_SECONDS=30
`
```

- 필수 설정값은 APP_ENV 하나이고, 나머지는 LLM 호출에 필요합니다.
- LLM_API_KEY 대신 OPENAI_API_KEY 를 써도 됩니다.
- DB 관련 값은 비워 뒀도 됩니다. 이 평가는 DB 를 건드리지 않습니다.
- 스크립트는 .env 가 있는 디렉터리에서 실행해야 설정이 잡힙니다.

4. 스크립트 배치

이 폴더의 ■■■■/*.py 를 backend/scripts/ 에 넣고,
■■■■/ 을 backend/evaluation_data/briefing/v1/ 로 넣습니다.

```
backend/  
scripts/  
expand_briefing_reports.py  
build_briefing_evaluation.py  
run_briefing_evaluation.py  
judge_briefing.py  
summarize_briefing.py  
check_briefing_evaluation.py  
evaluation_data/briefing/v1/  
timelines/ ← 이 폴더의 데이터셋/timelines  
cases/ ← 이 폴더의 데이터셋/cases (3단계에서 다시 만들 수도 있음)
```

5. 실행

(0) 보고서 본문 — 이미 만들어져 있어 다시 돌리지 않아도 된다

timelines/*/reports.json 의 seed 는 사람이 쓴 사실표이고 common_body.deals 는
그 사실만으로 실제 보고서 분량(딜 본문 800자 이상)으로 확장한 본문이다.

LLM 으로 만들었기 때문에 다시 돌리면 문장이 달라진다. **재현은 저장된 본문으로 한다.**

본문을 새로 만들어야 할 때만:

```
`bash  
.venv/bin/python scripts/expand_briefing_reports.py ₩  
--timeline evaluation_data/briefing/v1/timelines/hanbit
```

기준(사실표 수치 보존, 제작 용어·회차 메타·괄호 머리표 금지, 분량)을 통과한 보고서는
건너뛰고, 실패한 것만 다시 만든다.

(1) 타임라인 → 평가 케이스

```
`bash  
cd backend  
for t in hanbit sejong daeyang miraero hanul cheongsan; do  
.venv/bin/python scripts/build_briefing_evaluation.py ₩  
--timeline evaluation_data/briefing/v1/timelines/$t ₩
```

```
--out evaluation_data/briefing/v1/cases ₩
```

```
--depths 1 3 5 10 30
```

```
done
```

```
,
```

체인 6개 × 깊이 5개 = **케이스 30개**가 만들어집니다. LLM 을 쓰지 않는 단계라 즉시 끝납니다.

(2) 실제 브리핑 생성

```
`bash
```

```
.venv/bin/python scripts/run_briefing_evaluation.py ₩
```

```
--cases evaluation_data/briefing/v1/cases ₩
```

```
--out evaluation_results/briefing/run-001
```

```
,
```

30건에 약 15~20분, LLM 호출 30회. 한 건만 돌리려면 `--only hanbit-d30` 을 붙입니다.

(3) 채점

```
`bash
```

```
.venv/bin/python scripts/judge_briefing.py ₩
```

```
--cases evaluation_data/briefing/v1/cases ₩
```

```
--run evaluation_results/briefing/run-001 ₩
```

```
--out evaluation_results/briefing/run-001/judged
```

```
,
```

LLM 호출 30회.

(4) 검사 — 실행 전후에 돌린다

```
`bash
```

```
.venv/bin/python scripts/check_briefing_evaluation.py ₩
```

```
--root evaluation_data/briefing/v1 ₩
```

```
--run evaluation_results/briefing/run-001
```

```
,
```

`--run` 없이 돌리면 골든셋만 검사한다. 브리핑 실행 전에 반드시 통과시킨다.

LLM 을 쓰지 않는다.

(5) 집계

```
`bash
```

```
.venv/bin/python scripts/summarize_briefing.py ₩
```

```
--judged evaluation_results/briefing/run-001/judged ₩
```

```
--out evaluation_results/briefing/run-001/SUMMARY.json
```

```
,
```

LLM 을 쓰지 않습니다.

6. DB 없이 도는 이유

브리핑 Agent 는 스냅샷에 `_report_scope` 가 있으면 DB 를 조회하고, 없으면 스냅샷 값을 그대로 돌려줍니다. 케이스의 `input.json` 에는 이 키를 넣지 않습니다.

보고서 원문 재조회처럼 DB 로 가는 경로가 하나 남아 있어서, 하네스가 그 조회 함수만 스냅샷에서 답하도록 바꿔 끼웁니다(`run_briefing_evaluation.py`).

그래서 도구는 실제로 호출되면서 값만 고정됩니다. 도구를 꺾다면 못 켜줄 "필요할 때 과거 보고서를 찾아 읽는가" 까지 평가 대상으로 남습니다.

7. 결과가 달라질 수 있는 지점

- 같은 케이스라도 LLM 응답은 매번 조금씩 다릅니다. 절대 점수보다 **깊이별 경향** 을 봅니다.
- 브리핑 프롬프트 버전이 바뀌면 점수가 달라집니다. 실행 기록의 `prompt_version` 으로 어떤 버전이었는지 확인합니다. 이 결과는 `contract_management.generate_briefing.v13` 입니다.
- 첫 미팅(d01)은 LLM 을 타지 않고 고정 문구를 반환하므로 항상 같은 출력이 나옵니다.

8. 재현 확인 방법

`check_briefing_evaluation.py` 가 전부 통과하고, `■■■/SUMMARY.json` 과 비교해 깊이별 평균이 $\pm 5\%$ 안에서 움직이면 같은 조건으로 돌아간 것입니다.